

Programming in Java

Module-1

Section 2

Faculty Name – Dr. Vinod Jain

Syllabus Covered

Data Type, Operators, Control Structures: Variables, Constants, Declaration, Literals, Scope of Variable, Type Casting, Arithmetic Operators, Relational Operators, Logical Operators, Assignment Operator, Dot Operators, Control Statements.

Section 2

Data Type, Operators, Control Structures	Variables, Constants, Declaration, Literals, scope of variables
	Type Casting,
	Arithmetic Operators, Relational Operators
	Logical Operators, Assignment Operator, Dot Operators, Control Statements

Data Type

- A data type define the type of data and size of data.
- Data type is used to define and use variables and constants in programs.
- For example int, float, char float, double etc.

The Primitive Types

- The primitive types represent single values—not complex objects.
- Java defines **eight primitive types** of data: byte, short, int, long, char, float, double, and boolean.

Keyword	Description
byte	Byte-length integer
short	Short integer
int	Integer
long	Long integer
float	Single-precision floating point
double	Double-precision floating point
char	A single character
boolean	A Boolean value (true or false)

The Primitive Types

- The primitive types represent single values—not complex objects.
- Java defines **eight primitive types** of data: byte, short, int, long, char, float, double, and boolean.
- These can be put in **four groups**:
 1. Integers
 2. Floating-point numbers
 3. Characters
 4. Boolean

Integers

- Integers This group includes byte, short, int, and long, which are for whole-valued signed numbers.
- All of these are signed, positive and negative values.
- Java does not support unsigned, positive-only integers.
- The width and ranges of these integer types vary widely, as shown in this table:

Name	Width	Range
long	64	−9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
int	32	−2,147,483,648 to 2,147,483,647
short	16	−32,768 to 32,767
byte	8	−128 to 127

Floating-Point Types

- This group includes **float and double**, which represent numbers with fractional precision.
- Floating-point numbers, also known as **real numbers**, are used when evaluating expressions that require fractional precision.
- For example, calculations such as square root, sine and cosine, result in a value whose precision requires a floating-point type.

Name	Width in Bits	Approximate Range
double	64	4.9e−324 to 1.8e+308
float	32	1.4e−045 to 3.4e+038

Characters

- In Java, the data type used to store characters is **char**.
- This group includes char, which represents symbols in a character set, like letters and numbers.
- **Java uses Unicode to represent characters.**
- *Unicode* defines a fully international character set that can represent all of the characters found in all human languages.
- It is a unification of dozens of character sets, such as Latin, Greek, Arabic, Cyrillic, Hebrew, Katakana, Hangul, and many more.
- For this purpose, it requires 16 bits. **Thus, in Java char is a 16-bit type.**

Booleans

- Java has a primitive type, called **boolean**, for logical values. It can have only one of two possible values, **true** or **false**.
- This is the type returned by all relational operators, as in the case of `a < b`.
- `boolean` is also the type *required* by the conditional expressions that govern the control statements such as `if` and `for`.

```
boolean b;
```

```
b = false;
```

```
System.out.println("b is " + b);
```

Literals or Constants

- Literals are constants whose value is fixed.
- For example 10, 3.14, 1000, 'Y' , "Vijay Kumar" etc.
- Types of literals in java are

- 1. Integer Literals**

- 2. Floating-Point Literals**

- 3. Boolean Literals**

- 4. Character Literals**

- 5. String Literals**

Types of literals

Integer Literals

Eg. `Int x=10;` Here 10 is an integer literal.

Floating-Point Literals

Eg. `Float pi=3.14F;` Here 3.14 is a float literal.

Boolean Literals

Eg. `Boolean b=false;` Here false is a Boolean literal

Character Literals

Eg. `Char ch = 'Z';` here 'Z' is a character literal

String Literals

eg. `String name="Vijay";` Here "Vijay" is a string literal.

Variables

- The variable is the basic unit of storage in a Java program.
- Variables are used to store values. The value of a variable may change during execution of a program

Variables

- Variables are used to store input data, output data and intermediate results.
- Eg. In the program to calculate factorial of a number we declare
int n,i,fact;
- Here n is a variable used to store input value, fact is a variable to store output value and i is used to store intermediate results.

Variables

- A variable is defined by the combination of an identifier, a type, and an optional initializer.
- In addition, all variables have a scope, which defines their visibility, and a lifetime.

Declaring a Variable

- A variable declaration is a statement that reserves a named memory location and includes the following:
 1. A data type that identifies the type of data that the variable will store
 2. An identifier that is the variable's name
 3. An optional assignment operator and assigned value, if you want a variable to contain an initial value
 4. An ending semicolon
- *type identifier [= value][, identifier [= value] ...] ;*
- Eg. float price=99.99, interest_rate=8.25;
- int a=10,b=20;

Need of type casting

- When you perform arithmetic with variables or constants of the same type, the result of the operation retains the same type.
- For example, when you divide two ints, the result is an int, and when you subtract two doubles, the result is a double.
- Eg.
- `int x = 10+20;`
- `int y= x+1;`

Need of type casting

- Often, however, you might want to perform mathematical operations on operands with unlike types. It needs type casting.
- For example
- `int x=3,y=4;`
- `int sum=x+y;`
- `float average = sum/2;`
- OR
- `float average = (float)sum/2;`

Type Casting or Type Conversion

- **The process of converting one data type to another is type conversion or type casting.**
- Java performs some conversions for you **automatically** or **implicitly**, but other conversions must be requested **explicitly** by the programmer.

Automatic Type Conversion

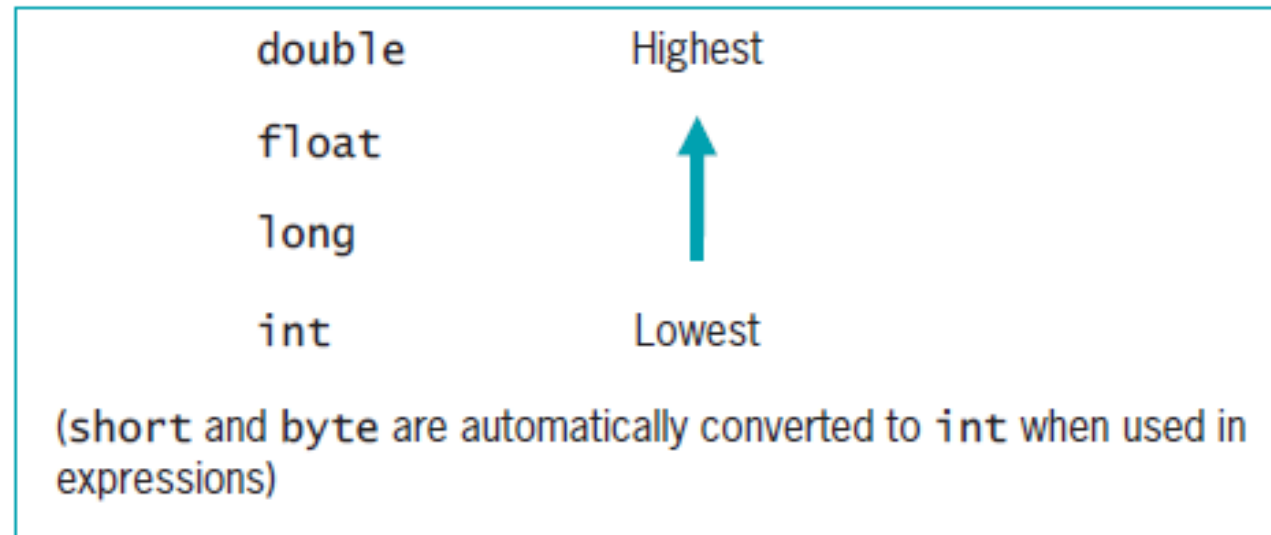
- When you perform arithmetic operations with operands of unlike types, Java chooses a **unifying type** for the result.
- The **unifying type** is the type to which all operands in an expression are converted so that they are compatible with each other.
- Java performs an implicit conversion; that is, it automatically converts nonconforming operands to the unifying type.
- Implicit conversions also are called promotions.

Automatic Type Conversion

- Implicit conversions also are called promotions. Figure shows the order for establishing unifying types between values.

Byte → Short → Int → Long → Float → Double

Widening or Automatic Conversion



Automatic Type Conversion

- For example, assume that an int, hoursWorked, and a double, payRate, are defined and then multiplied as follows:

```
int hoursWorked = 37;
```

```
double payRate = 6.73;
```

```
double grossPay = hoursWorked * payRate;
```

- The result of the multiplication is a double because when a double and an int are multiplied, the int is promoted to the higher-ranking unifying type double—the type that is higher in the list in Figure Therefore, assigning the result to grossPay is legal.

Invalid : Type Conversion

- The following code will not compile because hoursWorked times payRate is a double, and **Java does not allow the loss of precision** that occurs if you try to store the calculated double result in an int.

```
int hoursWorked = 37;
```

```
double payRate = 6.73;
```

```
int grossPay = hoursWorked * payRate;
```

Explicit Type Conversions

- **Explicit Type casting forces** a value of one data type to be used as a value of another type.
- To perform a explicit type cast, you use a cast operator, which is created by placing the desired result type in parentheses.
- Using a cast operator is an explicit conversion.
- `int x=3,y=4;`
- `int sum=x+y;`
- `float average = (float)sum/2; // It convert sum into float and then execute`

Operators

- Operators are used to perform operations such as addition, multiplication, comparison etc.
- Some operators are
- +, -, %, <, >, !=, ==, &&, |, & etc.

Types of Operators

- Arithmetic Operators
- Relational Operators
- Logical Operators
- Assignment Operator
- Dot Operators

Arithmetic Operators

- Arithmetic operators are used in mathematical expressions.
- The operands of the arithmetic operators must be of a numeric type.
- Eg.
- $X = 10 + 20;$

Arithmetic Operators

Operator	Result
+	Addition
-	Subtraction (also unary minus)
*	Multiplication
/	Division
%	Modulus
++	Increment
+=	Addition assignment
-=	Subtraction assignment
*=	Multiplication assignment
/=	Division assignment
%=	Modulus assignment
--	Decrement

The Basic Arithmetic Operators

Operator	Result
+	Addition
−	Subtraction (also unary minus)
*	Multiplication
/	Division
%	Modulus

Arithmetic Compound Assignment Operators

<code>+=</code>	Addition assignment
<code>-=</code>	Subtraction assignment
<code>*=</code>	Multiplication assignment
<code>/=</code>	Division assignment
<code>%=</code>	Modulus assignment

Arithmetic Compound Assignment Operators

- As you probably know, statements like the following are quite common in programming:

`a = a + 4;` is same as `a += 4;`

- This version uses the `+=` *compound assignment operator*.
- Both statements perform the same action: they increase the value of `a` by 4.

Increment and Decrement

Operator	Result
++	Increment
--	Decrement

Increment and Decrement

- The ++ and the -- are Java's increment and decrement operators.
- The increment operator increases its operand by one.
- The decrement operator decreases
- its operand by one. For example, this statement:

`x = x + 1;` can be rewritten `x++;`

- Similarly, this statement:

`x = x - 1;` can be rewritten `x--;`

Pre and Post increment or decrement

- `x = 42;`
- `y = ++x;`
- `Y=?`

Pre and Post increment or decrement

- `x = 42;`
- `y = ++x;`
- In this case, **y** is set to 43.
- Because the increment occurs *before* **x** is assigned to **y**.

Pre and Post increment or decrement

- `x = 42;`
- `y = x++;`
- `Y = ?`

Pre and Post increment or decrement

- `x = 42;`
- `y = x++;`
- the value of **x** is obtained before the increment operator is executed, so the value of **y** is 42.

Relational Operators

Relational Operators

Operator	Result
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

The outcome of these operations is a **boolean** value.

Relational Operators

- The *relational operators* determine the relationship that one operand has to the other.
- The outcome of these operations is a **boolean** value.
- The relational operators are most frequently used in the expressions that control the **if** statement and the various loop statements.

Example

```
public class operators1
{
    public static void main(String args[])
    {
        int a = 4;
        int b = 1;
        boolean c = a < b;
        System.out.print("\n c = " + c);
        System.out.print("\n\n");
    }
}
```

OUTPUT

run:

C = false

BUILD SUCCESSFUL (total time: 0
seconds)

Output

- run:
- C = false
- BUILD SUCCESSFUL (total time: 0 seconds)

Logical Operators

- Used to make logical expressions.
- Logical expressions may combine two or more relational expressions.
- The three commonly used logical operators are
 1. AND (&&)
 2. OR (||)
 3. NOT (!)

Logical Operators

Operator	Meaning	Work
&&	Logical AND	If both operands are true then only "logical AND operator" evaluate true.
	Logical OR	The logical OR operator is only evaluated as true when one of its operands evaluates true. If either or both expressions evaluate to true, then the result is true.
!	Logical Not	Logical NOT is a Unary Operator, it operates on single operands. It reverses the value of operands, if the value is true, then it gives false, and if it is false, then it gives true.

Logical OR (||)

- The logical OR operator is only evaluated as true when one of its operands evaluates true.
- If either or both expressions evaluate to true, then the result is true.

A	B	A B
0	0	0
1	0	1
0	1	1
1	1	1

Logical OR (||)

Logical Operators		
Operator	Description	Example
	OR	<pre>x=6 y=3 x==5 y==5</pre>

Logical OR (||)

Logical Operators		
Operator	Description	Example
	OR	x=6 y=3 x==5 y==5 Return False

Logical AND (&&)

- If both operands are true then only logical AND operator evaluate true otherwise it is evaluate false.

A	B	A&&B
0	0	0
1	0	0
0	1	0
1	1	1

Logical AND (&&)

Logical Operators		
Operator	Description	Example
&&	AND	x=6 y=3 x<10 && y>1

Logical AND (&&)

Logical Operators		
Operator	Description	Example
&&	AND	x=6 y=3 x<10 && y>1 Return True

Logical NOT (!)

- Logical NOT is a Unary Operator, it operates on single operands.
- It reverses the value of operands, if the value is true, then it gives false, and if it is false, then it gives true.

A	!A
0	1
1	0

Logical NOT (!)

Logical Operators		
Operator	Description	Example
!	NOT	x=6 y=3 !(x==y)

Logical NOT (!)

Logical Operators		
Operator	Description	Example
!	NOT	x=6 y=3 !(x==y) Return True

Logical Operators

Logical Operators		
Operator	Description	Example
&&	AND	x=6 y=3 x<10 && y>1 Return True
 	OR	x=6 y=3 x==5 y==5 Return False
!	NOT	x=6 y=3 !(x==y) Return True

Assignment Operator

- It is used to assign value to a variable
- The *assignment operator* is the single equal sign, =.
- It has this general form:

var = expression;

Eg. `x=10; //` where x is a variable

OR

`int x, y, z;`

`x = y = z = 100; //` set x, y, and z to 100

Dot Operator (.)

- The (.) operator is also known as member operator it is used to access the member of a package or a class.
- The dot operator links the name of the object with the name of a variable or method.

Dot Operator (.) : Example

```
public class Sample {
```

```
    void display() {
```

```
        double d = 20.3;
```

```
        int i = (int)d;
```

```
        System.out.println(i);
```

```
    }
```

```
    public static void main(String args[])
```

```
{
```

```
    Sample s = new Sample();
```

```
    s.display();
```

```
}
```

```
}
```

run:

20

BUILD SUCCESSFUL (total time: 4 seconds)

Here in statement

s.display()

The dot operator combines display() method with instance s.

Control Structures

- programming language uses *control* statements to cause the flow of execution to advance and branch based on changes to the state of a program.
- Java's program control statements can be put into the following categories:
 - Selection – If and Switch
 - Iteration - Loops
 - jump. – Break and continue

Java's Selection Statements

- Java supports two selection statements:
- **if** and **switch**
- Java supports two selection statements: **if** and **switch**

The if statement

- The **if** statement is Java's conditional branch statement.
- It can be used to route program execution through two different paths.
- Four types of if statement
 1. Simple if
 2. If-Else Statement
 3. Nested if
 4. If-Else Ladder

Simple if

- An **if statement** consists of a boolean expression followed by one or more statements.

Syntax

```
if(Boolean_expression)
```

```
{
```

```
    // Statements will execute if the Boolean expression is true
```

```
}
```

Simple if : Example

```
int age=20;  
//checking the age  
if(age>18){  
    System.out.print("Age is greater than 18");  
}
```

If-Else Statement

- An **if statement** can be followed by an optional **else statement**, which executes when the boolean expression is false.

```
if(Boolean_expression) {  
    // Executes when the Boolean expression is true  
}  
else {  
    // Executes when the Boolean expression is false  
}
```

If-Else Statement

```
int number=13;  
//Check if the number is divisible by 2 or not  
if(number%2==0){  
    System.out.println("even number");  
}else{  
    System.out.println("odd number");  
}
```

Nested if

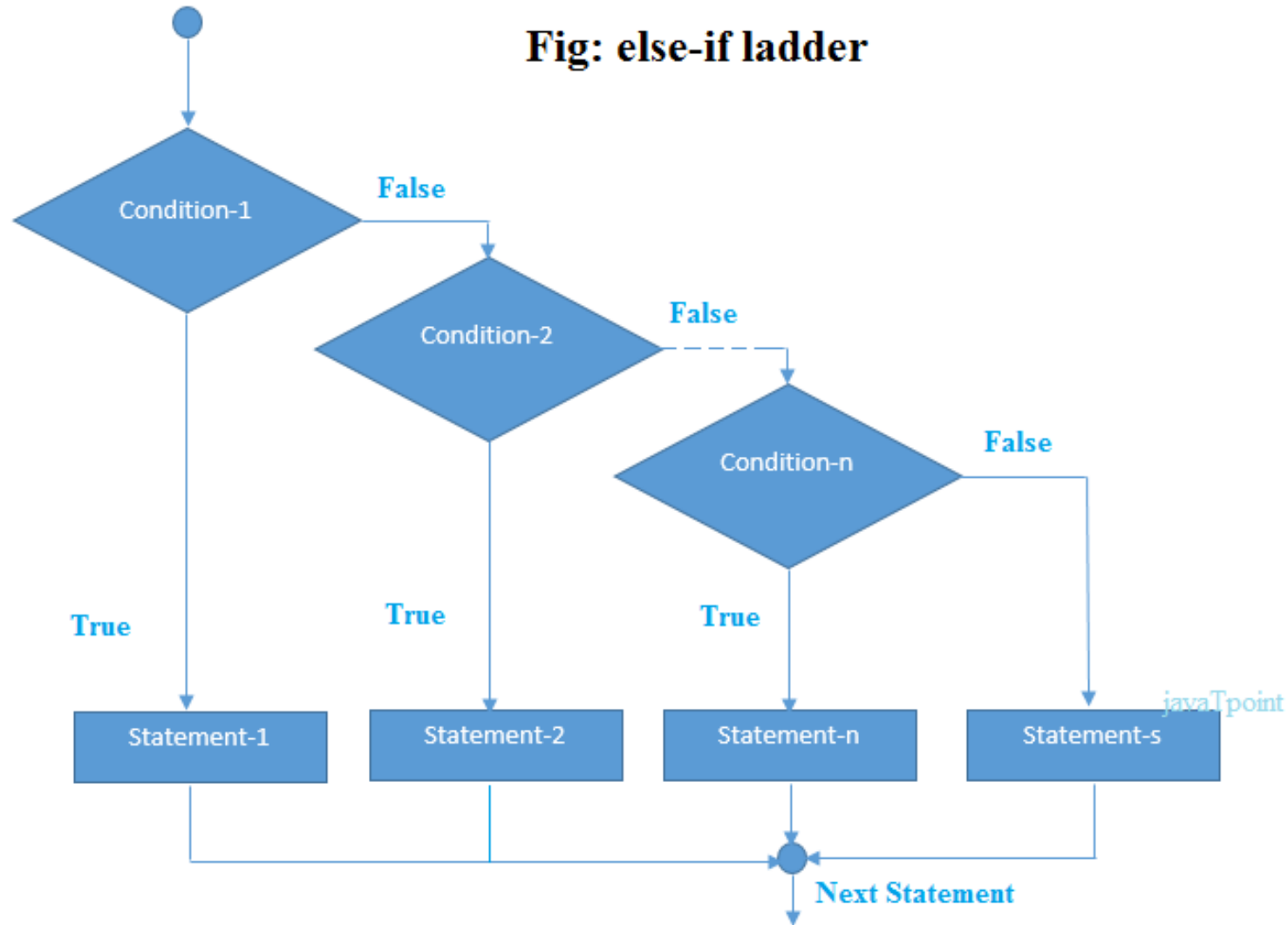
- It is always legal to nest if-else statements which means you can use one if or else if statement inside another if or else if statement.

```
if(Boolean_expression 1) {  
    // Executes when the Boolean expression 1 is true  
    if(Boolean_expression 2) {  
        // Executes when the Boolean expression 2 is true  
    }  
}
```


Nested if

```
int x = 30;  
int y = 10;  
if( x == 30 )  
{  
    if( y == 10 )  
    {  
        System.out.print("X = 30 and Y = 10");  
    }  
}
```

if-else-if ladder



if-else-if ladder

- The if-else-if ladder statement executes one condition from multiple statements.

```
if(condition1){  
    //code to be executed if condition1 is true  
}  
else if(condition2){  
    //code to be executed if condition2 is true  
}  
else if(condition3){  
    //code to be executed if condition3 is true  
}  
...  
else{  
    //code to be executed if all the conditions are false  
}
```

if-else-if ladder

```
int marks=65;
if(marks<50){
    System.out.println("fail");
}
else if(marks>=50 && marks<60){
    System.out.println("D grade");
}
else if(marks>=60 && marks<70){
    System.out.println("C grade");
}
```

```
else if(marks>=70 && marks<80){
    System.out.println("B grade");
}
else if(marks>=80 && marks<90){
    System.out.println("A grade");
}else if(marks>=90 && marks<100){
    System.out.println("A+ grade");
}else{
    System.out.println("Invalid!");
}
```

if-else-if ladder

- **Program to check POSITIVE, NEGATIVE or ZERO:**

```
int number=-13;  
if(number>0){  
    System.out.println("POSITIVE");  
}else if(number<0){  
    System.out.println("NEGATIVE");  
}else{  
    System.out.println("ZERO");  
}
```

Iteration - Loops

- There may be a situation when you need to execute a block of code several number of times.
- A **loop** statement allows us to execute a statement or group of statements multiple times.

Loops

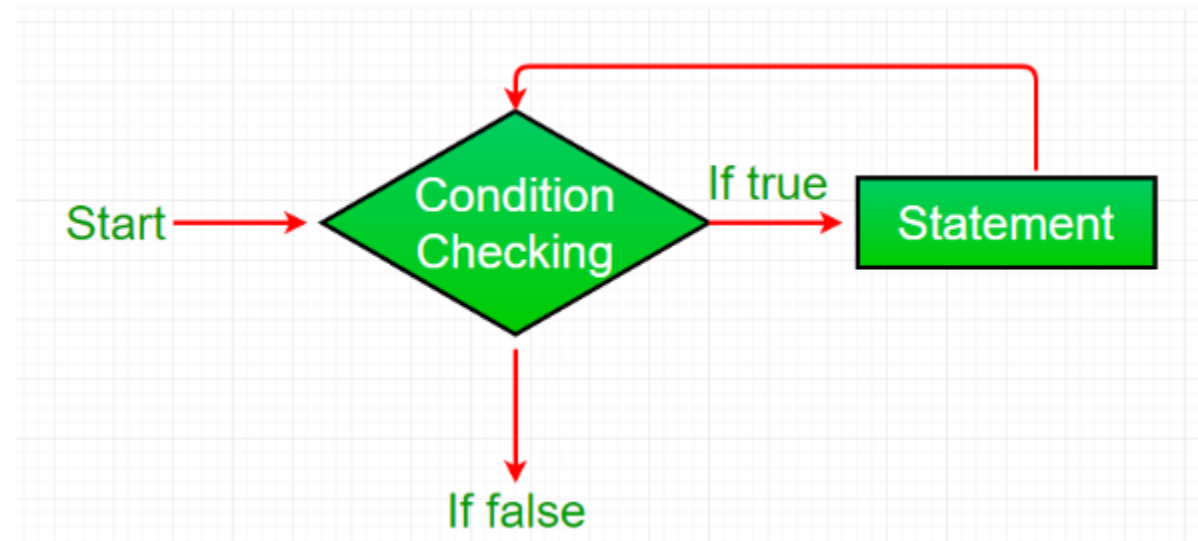
- Types of Loops
- For Loop
- While Loop
- Do-While Loop
- Enhanced For Loop

While Loop

- A while loop is a control flow statement that allows code to be executed repeatedly based on a given Boolean condition.

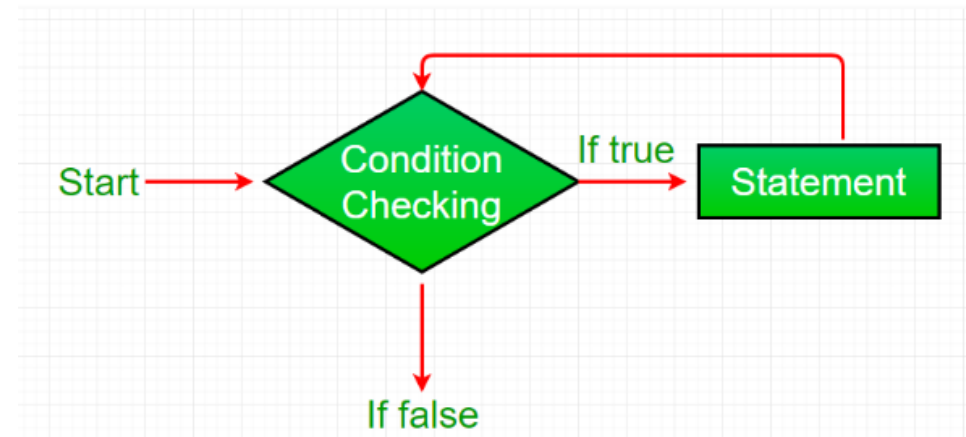
while (boolean condition)

```
{  
    loop statements...  
}
```



While Loop

- While loop starts with the checking of condition.
- If it evaluated to true, then the loop body statements are executed otherwise first statement following the loop is executed.
- For this reason it is also called Entry control loop
- When the condition becomes false, the loop terminates which marks the end of its life cycle.



while loop

```
// Factorial using while loop
```

```
int n=4,i,fact=1;
```

```
i=1;
```

```
while(i<=n)
```

```
{
```

```
    fact=fact*i;
```

```
    i++;
```

```
}
```

```
System.out.print("\n Fact = " + fact);
```

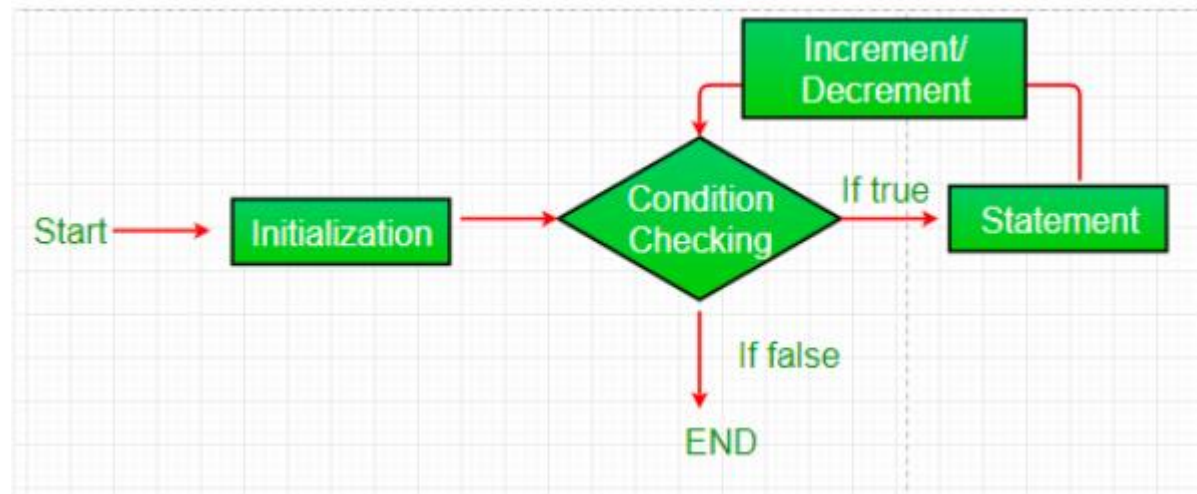
For loop

- A for statement consumes the initialization, condition and increment/decrement in one line thereby providing a shorter, easy to debug structure of looping.

Syntax:

for (initialization condition; testing condition; increment/decrement)

```
{  
    statement(s)  
}
```



For Loop

// Factorial using for loop

```
int n=4,i,fact=1;
```

```
for(i=1;i<=n;i++)
```

```
{
```

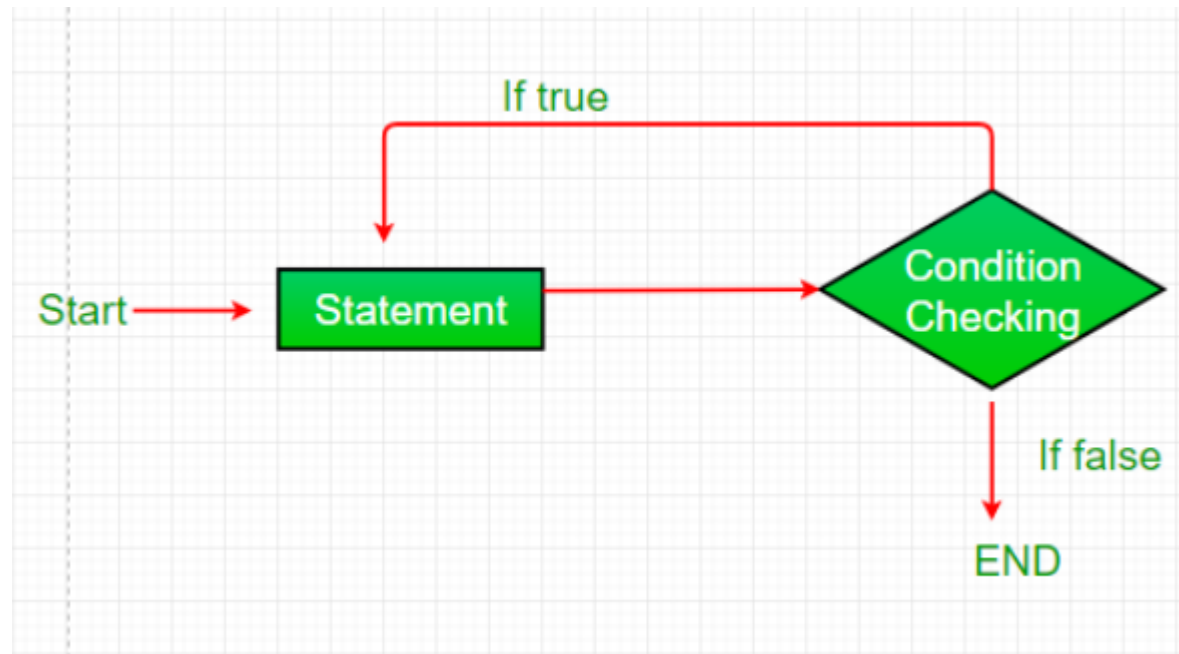
```
    fact=fact*i;
```

```
}
```

```
System.out.print("\n Fact = " + fact);
```

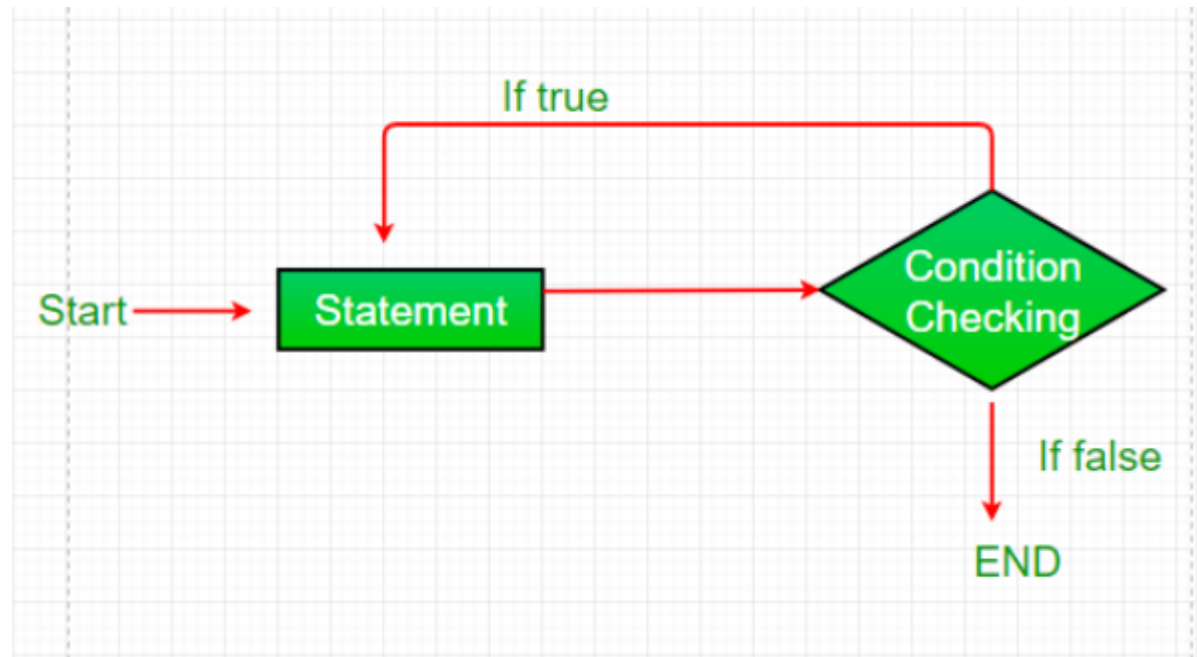
do while loop

- It is similar to while loop.
- Only difference that it checks for condition after executing the statements, and therefore is an example of Exit Control Loop.



do while loop

```
// Factorial using do while
int n=4,i,fact=1;
i=1;
do
{
    fact=fact*i;
    i++;
}while(i<=n);
System.out.print("\n Fact = " + fact);
```



Enhanced For loop

- Enhanced for loop provides a simpler way to iterate through the elements of a collection or array.

Syntax:

```
for (T element:Collection obj/array)
{
    statement(s)
}
```

Enhanced For loop

```
for (T element:Collection obj/array)
{
    statement(s)
}
```

Example

//enhanced for loop Example

```
String array[] = {"Ron", "Harry", "Hermoine"};
for (String x:array)
{
    System.out.println(x);
}
```